

Pomona: Continuous Code Quality Improvement via Small, Agentic Pull Requests at Bloomberg

David Williams
University College London
London, United Kingdom
david.williams.22@ucl.ac.uk

Angelos Evripiotis
Bloomberg
London, United Kingdom
jevripiotis@bloomberg.net

Serkan Kirbas
Bloomberg
London, United Kingdom
skirbas@bloomberg.net

Sergey Magidovich
Bloomberg
London, United Kingdom
smagidovich@bloomberg.net

Harry Morgan
Bloomberg
London, United Kingdom
hmorgan39@bloomberg.net

Peter Wainwright
Bloomberg
London, United Kingdom
pwainwright@bloomberg.net

Federica Sarro
University College London
London, United Kingdom
f.sarro@ucl.ac.uk

Abstract—In this industrial experience paper, we present *Pomona*, a lightweight agentic tool that utilises agent skills for continuous code quality improvement. Inspired by the KaizenTM philosophy, *Pomona* automates a cycle of discovery and incremental repair: a *Scanning* skill identifies tasks and prioritises them in a backlog, while a *Repair* skill generates small, easily reviewable pull requests (PRs). This design enables frequent, low-risk improvements while maintaining engineer trust and reducing technical debt. We evaluated *Pomona* at Bloomberg through a three-month team deployment and a questionnaire distributed to senior engineers. The results are promising: 32/39 PRs (82.1%) were merged with a median time-to-close of just over two hours. Moreover, 10/12 surveyed engineers expressed a desire to adopt *Pomona*, praising its small diff sizes and focus on code quality. Since our evaluation, another team has adopted *Pomona*. We conclude with actionable insights for deploying agents in industry.

Index Terms—Software Quality, Technical Debt, AI4SE, Agent

I. INTRODUCTION

As Large Language Model (LLM) integrations and agentic solutions [1]–[3] mature in the domain of Software Engineering (SE), organisations are increasingly looking to implement reliable techniques to improve the software development process for their employees. Although some studies on the impact of LLM-based SE tools on developer productivity have reported significant increases in development speed [4], others have observed that developers using LLMs tend to perform their regular tasks more slowly than without (while believing they are working faster) [5]. Notably, He et al. [6] found that agent adoption in open source repositories increases development velocity only temporarily and comes at the cost of long-term increases in code complexity and quality issues.

To reconcile this tension between short-term speed and long-term maintainability, we present a novel, KaizenTM-inspired¹ agentic solution, dubbed *Pomona*, that utilises agent skills to improve code quality continuously through small, automated

changes. Agent skills are user-defined instructions that enable autonomous agents to handle specialised, repeatable workflows [8], [9]. *Pomona* is composed of two such skills: one for identifying code quality improvement tasks and another for addressing them. It builds and maintains a structured, prioritised backlog of these tasks, which it gathers from multiple sources, e.g., static analysers, inline technical debt markers, test coverage gaps, and deviations from project-specific coding standards. Picking the highest-priority item from the backlog each time, *Pomona* generates tiny pull requests (PRs) targeting approximately 10 lines of diff, then updates the backlog and repeats the process. *Pomona* handles the needs and challenges of efficiently adopting AI by being ① low friction, in that the task discovery and patch generation process are autonomous background workflows that do not require engineers to adapt their development activities; ② low stakes, adopting a human-in-the-loop review paradigm so that *Pomona*’s contributions cannot affect the codebase without receiving engineer approval (with that process made especially simple for the engineer given the small size of *Pomona*’s PRs); and ③ focused on improving code quality, ensuring that changes introduced by *Pomona* are guaranteed to benefit the long-term maintainability of the targeted codebase.

Pomona is already in use at Bloomberg, with positive impressions. To evaluate its potential value in practice, we conducted an early mixed-methods study. Starting with a manual audit of 39 PRs generated by *Pomona* for the first adopting team over the course of a three-month deployment, we observed both high acceptance and efficiency: 32/39 (82.1%) of the generated PRs were successfully merged, and 25 of those accepted required no human interaction beyond the mandatory review, all within a median time-to-close of 2 hours and 14 minutes. We complement these findings with insights from a questionnaire distributed to 12 senior engineers not yet using *Pomona*. The results are promising, with 10/12 participants expressing a desire to try the tool, particularly valuing *Pomona*’s small diff sizes (10/12) and focus on improving code quality (10/12). At the time of

¹KaizenTM is a Japanese word and business methodology promoting continuous improvement through incremental, consistent changes [7].

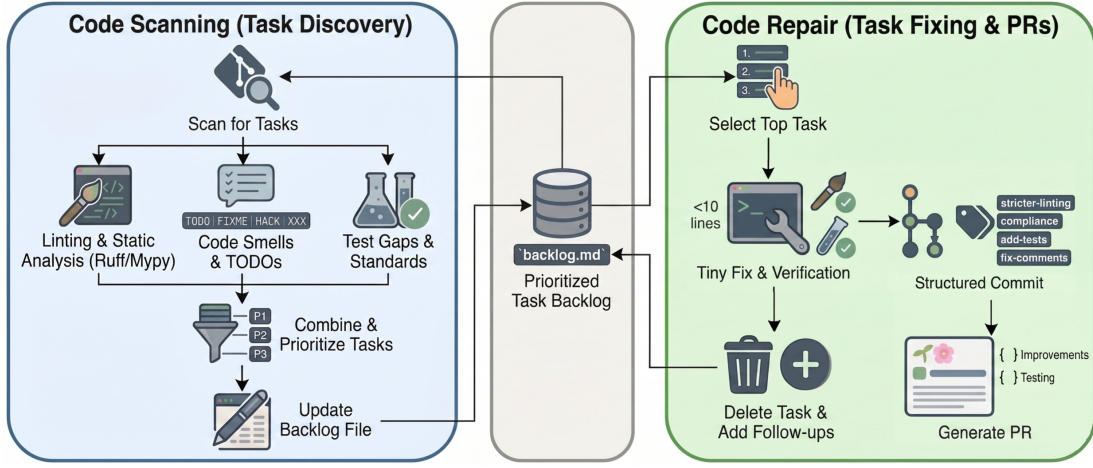


Fig. 1. Overview of Pomona. Each shaded region represents the responsibilities of an agent skill (left: Scanning; right: Repair).

writing, a second engineering team has adopted Pomona, and plans are in place to distribute it more widely within Bloomberg. In the remainder of this paper, we provide an in-depth description of Pomona’s implementation, and offer insights into practical strategies for deploying agentic AI4SE in a large-scale industrial environment.

II. POMONA OVERVIEW

Figure 1 illustrates an overview of Pomona, which comprises two agent skills: one for identifying code quality improvement tasks in the target repository and compiling them in a structured, prioritised backlog (the Scanning skill, §II-B); and another for selecting the highest-priority task in the backlog and generating a new PR that addresses it for human review (the Repair skill, §II-C). Leveraging the agent skills medium makes Pomona’s implementation simple: the Repair skill, the Scanning skill, and the structured task backlog are each implemented as individual Markdown files that can be integrated with any current agentic product. This format also makes Pomona programming-language-agnostic (though our specific implementation, evaluated in §III-A, targets Python). The following sections describe each component and their cyclical combination to achieve a continuous code-quality improvement loop.

A. Prioritised Task Backlog

In Pomona, a structured backlog of code quality improvement tasks lies at the centre of the Scanning and Repair skills. The Scanning skill fills the backlog with prioritised improvement tasks, while the Repair skill consumes these tasks to generate new PRs with improvements for review. An excerpt from the backlog is shown in Figure 2.

Backlog improvement tasks are categorised in priority groups P1–P4 defined in the “benefit $\times$ ease-of-review” matrix in Table I. By altering the specification of the Scanning skill, engineers adopting Pomona can define what constitutes *high benefit* in their development context. A good rule of thumb is

that bug-catching improvements should take priority over cosmetic ones. For example, a rule that prevents mutable default parameters is more valuable than one that reorders imports. For the specific implementation of Pomona we evaluated in §III-A, we included the following in our Scanning skill specification: “*high-benefit* changes catch real bugs or prevent future bugs (mutable defaults, loop variable capture, missing exception chains); reduce maintenance burden (removing dead code, fixing misleading comments); or improve developer experience (better error messages, clearer code)”.

Following a similar process to the one described above, adopting engineers can also define what counts as *easy to review*. For our implementation, we included the following rule in our Scanning skill: “A change is easy to review if it is fully automated (auto-fix, formatting), mechanical and repetitive (reviewer can scan quickly), small in scope (single file or rule) and does not introduce behavioural changes”.

Our initial design for Pomona stores the task backlog in a Markdown file in the repository root. While this design provides some benefits (i.e., the backlog is centralised and versioned alongside the codebase), we note that the backlog could also be implemented using project management software such as Jira via Model Context Protocol (MCP) integrations.

B. Scanning: Identifying Repair Tasks

The Scanning skill enables Pomona to automatically discover new code quality improvement tasks. It is executed as part of the Repair skill (§II-C), specifically when the backlog runs low on high-priority tasks (the P1 and P2 categories are empty). To refill the backlog, the Scanning skill initialises multiple sub-agents in parallel, each exploring a different discovery area. As described below, the quantity and purpose of sub-agents should be defined by the adopting engineer based on their needs. Once all sub-agents complete their searches, their outputs are combined into prioritised backlog tasks.

In our specific implementation of Pomona, three unique sub-agents are defined. Sub-agent 1 performs rule expansions for `ruff` (a Python linter and code formatter [10])

TABLE I
PRIORITY MATRIX FOR TASK MANAGEMENT

	Easy to review	Hard to review
High benefit	P1 — Do first	P3 — Worth doing, plan the split
Low benefit	P2 — Quick wins	P4 — Parking lot (don't pick)

P1: High benefit, easy to review

- Remove unused `min_lead_in / min_lead_out` params from `lead_time_stat_timeseries_repos` + 2 callers
- Remove self-documenting TODOs in `utils/agg.py` (lines 35, 45)
- Enable ruff `I` (isort) incrementally — all auto-fixable
 - `core/` (11 modules, strictest-checked — start here)
 - `utils/`
 - `pdata/`
- Enable ruff `UP` (pyupgrade) for `core/` — most rules auto-fixable

Fig. 2. Excerpt of Pomona’s backlog for the adopting Bloomberg engineering team featured in §III-A.

and strict type-checking expansion for mypy (a static type checker for Python [11]). Sub-agent 2 searches the codebase for TODOs expressed as “TODO | FIXME | HACK | XXX”, and classifies each finding according to the following guide: self-documenting removals (where the code is already done) are assigned as P1, clear fixes with obvious solutions are P1/P2, fixes requiring further domain investigation are P3, and aspirational comments are assigned as P4. Sub-agent 2 also searches for comments that contradict their surrounding code, as well as identifies dead code by searching for unused imports, commented-out code blocks (>3 consecutive commented lines), and unreachable code after early returns. Finally, sub-agent 3 identifies test coverage gaps by comparing source modules with unit test modules to reveal untested modules. It prioritises pure-logic modules without data dependencies (as these are easiest to test) and modules with complex branching logic (as these provide the highest value). In addition, sub-agent 3 checks compliance with the repository’s defined coding standards (usually found in agent configuration files) and looks for long functions (i.e., longer than 50 lines) that can be decomposed into smaller parts.

Once all agents complete their searches, the *Scanning* skill concludes by aggregating their findings. This aggregation is done in two steps: First, the findings are listed and concatenated, and duplicate findings are removed. Second, each unique finding is assigned to a priority category following the priority matrix shown in Table I and rules described in §II-A.

Now with priority levels assigned, tasks are converted to follow the backlog item format (shown in Figure 2) and added to their corresponding priority category. At this point, these tasks are ready to be addressed by the *Repair* skill, described in the following section.

C. Repair: Solving Tasks Identified by Scanning

The *Repair* skill is Pomona’s point of entry, implemented as an agent skill intended to be either executed by an engineer or configured to trigger periodically (e.g., once per day).

The first step of the *Repair* skill instructs the agent to access the task backlog at the root of the target repository. If the backlog file does not exist or the high-priority (P1 and P2) task categories are empty, the *Scanning* skill is executed first, after which the agent returns to complete the remaining steps in the *Repair* skill.

When there are tasks in the backlog, the next step is to *select the first task from the highest-priority category*. If an item has sub-items (as in Figure 2), the instructions specify to select the first sub-item.

The next step is to *implement the fix*. For this, a few rules are defined. First, any changes should be validated with the project’s testing and linting commands (as specified by the adopting team). Second, changes must be small: since the goal is to create tiny, easily reviewable PRs, the agent should aim for roughly 10 lines of diff. If a change exceeds that, the agent is instructed to split it using strategies provided by the adopting team. For example, in our implementation, one strategy is to enable linting rules/add tests for one directory at a time, and then add follow-up items for other directories to the backlog.

Once the code changes are made and validated, the agent *updates the backlog* by deleting the completed task and adding follow-up tasks (if any). Since the backlog is stored in the repository, there is no need to maintain a separate log of completed tasks, as the project’s `git` history tracks changes made to the backlog over time.

With the code and backlog changes complete, the agent can now *commit the changes*, including a clear motivation for the change in the commit body and links to relevant resources to help human reviewers better understand these automated changes when reviewing the project’s history down the line.

The final step of the *Repair* skill *asks the user whether they want to generate a PR*. If so, the agent creates one (our implementation uses Bloomberg’s internal MCP integrations to achieve this) following the repository’s standard PR template. This choice is optional depending on the control paradigm: if the skill is autonomously triggered periodically (e.g., once per day) rather than initialised by an engineer, this user choice is skipped, and a PR is created as soon as the previous steps are complete. The PR title is generated as a one-sentence summary of what was improved and why it matters, prefixed with specific emojis to clearly indicate that the PR is AI-generated. For the PR description, the agent is instructed to lead with the concrete outcomes for engineers. It only includes additional sections if they add value beyond the title. For example, it would include the “Improvements” section only if the title alone does not sufficiently explain the changes, and the “Testing & Linting” section only if special testing was done beyond the project’s standard testing commands.

III. EARLY INSIGHTS AND EVALUATION

We conducted a mixed-methods evaluation comprising an analysis of PRs generated by Pomona for one team over a three-month adoption period and a questionnaire distributed to non-Pomona users gauging their interest in its concept.

A. Analysing Pomona Pull Requests

We assessed Pomona’s early-stage impact by analysing PRs generated by Pomona for the first adopting team over a three-month period. During this time, Pomona generated 39 PRs, all of which were closed by the team (either merged or rejected). We manually extracted standardised metadata for each PR, including the resolved task’s priority level (P1–P4) and nature (i.e. source). To measure the complexity of each PR, we recorded the diff size (files and lines of code) and whether the changes led to additional subtasks being added to the backlog. Finally, we observed the outcomes and review dynamics for each PR, including whether the PR was merged, the time between creation and closing, the number of reviews and non-review comments, and whether any follow-up commits were required before merging. For rejected PRs, we noted any explicit reasoning provided.

1) *Results:* Our evaluation period spans approximately three months from mid-March to mid-June 2026. The team preferred to retain full control over the volume of AI-generated PRs in their repository, so a team member manually executed the `Repair` skill for each new PR. The team aimed for a goal of “one Pomona PR per day”, which they largely achieved, producing 39 PRs across 55 workdays (accounting for public and team holidays). Across these PRs, we observed a strong overall acceptance rate of 82.1% (32/39).

Regarding backlog priority levels, 31 PRs generated by Pomona were P1 and the remaining 8 were P2. Although tasks existed in the other priority-level groups, there were sufficient P1- and P2-level tasks such that the other priority groups weren’t reached within the three-month evaluation period. The majority (26/39) of the Pomona PRs addressed `ruff` rule violations, and other tasks included adding unit tests (5 PRs), removing dead code (3 PRs), and other repository-specific tasks (5 PRs). Notably, 6 PRs were directly requested by a team member, who manually added the corresponding backlog entries in different PRs. This example highlights the flexibility offered by Pomona’s accessible backlog in the repository root.

Taking a closer look at the seven rejected PRs reveals that four were due to “race conditions”, in which Pomona was executed twice before a human review, resulting in duplicate PRs. Each duplicate was rejected, while the original was accepted. Taking this oversight into account, we revised Pomona to skip tasks that are already addressed by currently open PRs. Of the remaining three rejections, two were due to changes being made with undesirable formatting, and the final one was closed without comment from the team.

Of the 32 accepted PRs, 28 were accepted as-is without requiring additional commits from the human reviewers, and 25 were accepted with no comments or interactions beyond the mandatory human review before merging. This largely positive outcome suggests that the scope of tasks tackled by Pomona was manageable enough to typically warrant only a brief review. Reflecting this low complexity, the median time to close (either merge or reject) for Pomona PRs was only 2 hours and 14 minutes. PR turnaround was consistently

swift: 25 of the 39 PRs (64.1%) were resolved within 4 hours, increasing to 29 (74.4%) within the same day and 33 (84.6%) by the next business day. While we do observe some outliers at both ends, ranging from 1 minute when duplicate PRs were closed to a maximum of four business days during periods where the team’s reviewing capacity was particularly low, the overall distribution of time-to-close measurements supports our claim that Pomona is low-friction and integrates into existing workflows without significant cognitive overhead.

Pomona PRs exhibited a substantial range in size. All PR size metrics (files/lines changed) exclude backlog changes to focus solely on codebase impact. The median number of files changed was 4 (mean 6.5), with a minimum of 1 and a maximum of 35. One potential cause for concern was the number of lines changed, calculated as the sum of added and deleted lines. The median was 37, the mean was 57.7, and values ranged from 5 to 168. While our original implementation of the `Repair` skill explicitly specified 10-line diffs, the agent sometimes failed to adhere to this limit when handling repetitive refactoring tasks. Despite this deviation, the team merged more than half of the nine PRs over 100 lines within the same business day, indicating that even larger-scale PRs can be manageable when the targeted improvements are intuitive. Here, larger PRs typically applied a single `ruff` rule to an entire file or subdirectory at once. Based on these empirical insights, the adopting team has since increased Pomona’s PR size limit to 100 lines and enforced it with a deterministic check that automatically rejects PRs exceeding that limit. We highlight the need for better proxies for estimating PR review effort as an important direction for future work.

Finally, six of the accepted Pomona PRs added new tasks to the backlog, including one that triggered the `Scanning` skill, which added 8 new tasks. The other five PRs added subtasks directly related to the task at hand to ensure the scope of changes did not become too large to review easily.

B. Non-User Questionnaire

To gauge interest in Pomona’s concept, we created a questionnaire to distribute among engineers. We specifically targeted engineers in Bloomberg’s Developer Experience (DevX) teams, given their rich understanding of the organisation’s development tooling. The questionnaire contained 12 questions, the first five of which covered participant demographics, including job title, years of experience, and familiarity with AI4SE tools. Next, the questionnaire presented the participant with Figure 1 alongside a high-level description of Pomona, followed by seven questions concerning whether the tool seems useful (and if not, why), how many pull requests the participant would be willing to review per week if they adopted the tool, which features seem most valuable, what types of tasks they would want Pomona to tackle, and concluding with an opportunity to provide any additional feedback to refine the concept. We provide the questionnaire as part of our supplementary materials [12].

1) *Results:* In total, 12 participants completed the questionnaire. This cohort comprised two Team Leads, nine Senior Software Engineers and one software engineer, of whom seven had more than 10 years of experience, four had 4–10 years of experience, and one had 1–4 years of experience. In terms of experience with AI-assisted software development (i.e., in-IDE tools), nine stated they use these tools daily, one weekly, another monthly, and one never. Similarly, six participants stated they use agentic tools (e.g., automated PR generation and review) daily, two weekly, three monthly, and one never.

When presented with Pomona’s concept, the majority (9/12) of participants felt the tool would be useful, and more (10/12) stated they wanted to try Pomona in future. If they adopted Pomona, most participants (8/12) would be willing to review 2–3 PRs per week, which is only a little less than the number observed in the adopting team described in §III-A (3–4 PRs per week). The remaining responses were split: one participant was willing to review more than 5 PRs per week, another would review 4–5 per week, and the remaining two preferred 1 or fewer per week. When asked about the most appealing features to them among those described in the concept, small diffs and focus on code quality improvement were both praised by 10/12 participants, while 7/10 placed importance on the automated identification and completion of tasks. These responses suggest most participants appreciate Pomona’s format but require high confidence and control over the automated task selection and prioritisation process.

In terms of tasks the participants wish Pomona would tackle, six requested handling of TODOs/outdated comments, six mentioned static analysis/linting issues, three mentioned simple bug fixing, and another three dependency management, seemingly because they trust agents to handle issues of that scope autonomously. Others mentioned documentation, adding tests (which would be the *“one exception to the size [limit] of the PR”*), performance fixes, and removing dead code.

A few participants also highlighted potential drawbacks and areas for future improvement. They highlighted the risk of review overload, fearing that Pomona might simply *“create more open PRs for actual humans to handle”*, or generate *“gratuitous churn”* by addressing tasks that, while technically valid, are *“not relevant enough to care about”*. Another participant noted that *“sometimes code smells require large-scale refactoring which seems to be out of scope [for Pomona]”*. However, the fact that 10/12 participants still expressed a desire to try the tool suggests that, for many, the aforementioned code quality improvement tasks may be smaller in scale but are primarily neglected due to a lack of engineering bandwidth rather than a lack of perceived value. In other responses, trust and agency emerged as key topics of interest. Several participants were wary of *“trust[ing] the AI judgement to discover the right issues to fix”* and expressed a preference for a user-controlled paradigm (i.e., a fully user-triggered Repair skill, as described in §III-A) over an autonomous triggering system, with this also extending to wanting *“control over what gets prioritised first”*. Finally, one respondent requested the ability for Pomona to iterate upon its own PRs based

on reviewer feedback. We consider agentic PR refinement a promising area for future work.

IV. DISCUSSION

Based on our experience of deploying and evaluating Pomona at Bloomberg, we discuss actionable insights into industrial adoption of agentic AI and outline future work for both researchers and practitioners.

Small Changes Facilitate Adoption. Pomona’s early success was not in solving the most “difficult” coding problems, but in solving high-priority, low-complexity tasks with high accuracy and minimal friction. The high PR acceptance rate in our preliminary real-world deployment suggests that current agents can confidently tackle “low-hanging fruit” in software repositories (e.g., linting violations, dead code). Small, intuitive changes reduce both reviewer scepticism and cognitive load. In fact, despite Pomona’s strong performance, the adopting team intentionally limited its output to at most 1 PR per day, indicating that human review bandwidth is a far greater bottleneck in AI-driven development than code-generation speed. Moreover, 83% of survey participants cited small diffs as Pomona’s most appealing feature, highlighting the importance of code reviewability in accepting AI contributions.

AI for Technical Debt Management. Our findings suggest that engineers want AI to handle “neglected” tasks (TODOs, outdated comments, dead code, and linting) because while these tasks are important, engineers struggle to find time to address them manually. Pomona demonstrates that agents can easily be configured to target TODOs, outdated comments, and dependency updates. These are the specific areas where engineers feel a strong “value-to-effort” gain from automation. The fact that several Pomona PRs either generated further tasks also shows that AI can act as a catalyst for ongoing technical debt management rather than just a one-off tool.

Prioritisation is as Important as Detection. While static analysis tools can identify many potential tasks, selecting the right improvements to apply is crucial. Our benefit $\times$ effort prioritisation strategy (Table I, §II-A) can focus agents on high-benefit, low-friction changes, ensuring that automated changes deliver tangible value.

Expert Scepticism and Human-in-the-Loop. Through Pomona’s PR-driven format, engineers retain full control over whether any changes are integrated into the codebase. These PRs clearly communicate the purpose and impact of each modification, following consistent templates that prioritise clarity and benefit for the engineer. This transparency is critical for building trust in automated changes. Our survey reveals that experienced engineers (10+ years’ experience) remain wary of “agentic judgment” in prioritising tasks. As a result, current agentic solutions should continue to promote a user-centric [13] paradigm (such as that of Pomona, with its user-initiated workflow, accessible backlog, and review-dependent impact), while one potential avenue for future research could be in automated task relevance filtering. Developing techniques that can distinguish “relevant” or “important” code changes from merely “valid” ones is a high-priority frontier.

Agentic Task Decomposition. Another direction for future work to facilitate industry AI adoption would be to investigate how agents can reliably identify and break down large tasks into small, easily reviewable ones without human intervention.

Quantifying Cognitive Load. Currently, it is difficult to identify the threshold at which agentic PRs transition from helpful suggestions to “gratuitous churn”. Future research should shift from binary “merge/reject” rates toward multi-dimensional metrics (including subjective engineer perceptions) that capture the cognitive load of human-AI collaboration. This effort requires devising proxies to measure the context-switching cost of reviewing AI and human code, as well as the mental effort involved in verifying different automated changes.

V. RELATED WORK

Preliminary research on agent skills in different domains has explored their effectiveness in improving task performance, including SE tasks [14], [15], and how to organise [16], secure [17], [18], and evolve skills at scale [19], [20]. The results generally show that curated skills can improve performance [14], especially in specific scenarios [15], although the gains in SE tasks remain context-dependent [14] and their overall impact is still unclear [15]. These results show the challenges of designing effective, reusable procedural knowledge for SE tasks, and the need for governance and validation mechanisms when deploying such skills in practice [17], [18].

In contrast to previous work, which primarily focuses on evaluating, generating, or securing agent skills, our work investigates how to devise and integrate agent skills in a real-world industrial setting based on user needs. The skills we present enable low-friction, continuous, incremental code-quality improvement, and we highlight its value to users through a preliminary study. In addition, we report practical lessons from its deployment.

VI. CONCLUSION

This work introduces *Pomona*, an agentic tool that leverages agent skills for continuous code quality improvement through automated, easily reviewable changes. Our preliminary evaluation, including a three-month team deployment and a questionnaire with senior practitioners, demonstrates *Pomona*’s potential to help manage technical debt with minimal impact on development velocity, and to serve as a low-stakes, high-benefit approach to agentic AI adoption in SE.

DATA AVAILABILITY STATEMENT

As *Pomona* has been developed and evaluated within Bloomberg’s internal systems and repositories, we cannot release the data and source code due to non-disclosure agreement policies. We also cannot reveal which AI services or models were used. However, *Pomona*’s implementation consists of three Markdown files (the *Repair* skill, the *Scanning* skill, and the *backlog*) and can be combined with any language model. In this work, we have detailed how to implement such a tool in practice, enabling academics and practitioners alike to adopt it in the future. Moreover, we make the questionnaire we distributed for user evaluation available [12].

REFERENCES

- [1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: agent-computer interfaces enable automated software engineering,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS ’24, vol. 37. Red Hook, NY, USA: Curran Associates Inc., Dec. 2024, pp. 50 528–50 652.
- [2] H. Ruan, Y. Zhang, and A. Roychoudhury, “SpecRover: Code Intent Extraction via LLMs,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ACM Conferences, Apr. 2025, pp. 963–974. [Online]. Available: <https://dl.acm.org/doi/10.1109/ICSE55347.2025.00080>
- [3] X. Wang *et al.*, “Openhands: An open platform for AI software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=OJD3ayDDoF>
- [4] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” Feb. 2023. [Online]. Available: <https://arxiv.org/abs/2302.06590v1>
- [5] J. Becker, N. Rush, E. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” Jul. 2025, arXiv:2507.09089 [cs]. [Online]. Available: <http://arxiv.org/abs/2507.09089>
- [6] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, “Speed at the cost of quality: How cursor ai increases short-term velocity and long-term complexity in open-source projects,” 2026. [Online]. Available: <https://arxiv.org/abs/2511.04427>
- [7] M. Imai, *Kaizen: The Key To Japan’s Competitive Success*. McGraw-Hill Education, 1986.
- [8] Github, Inc., “About agent skills,” <https://docs.github.com/en/copilot/concepts/agents/about-agent-skills>, 2026, accessed: July 5, 2026.
- [9] Anthropic PBC, “Skills — agent skills,” <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2026, accessed: July 5, 2026.
- [10] Astral, “Ruff,” <https://docs.astral.sh/ruff/>, 2026, accessed: July 5, 2026.
- [11] Mypy Project Contributors, “mypy - optional static typing for python,” <https://mypy-lang.org/>, 2026, accessed: July 5, 2026.
- [12] D. Williams *et al.*, “Pomona supplementary materials repository,” <https://figshare.com/s/09a23c22550f3e3967b2>, 2026, accessed: July 5, 2026.
- [13] D. Williams, J. Callan, S. Kirbas, S. Mehtaev, J. Petke, T. Prideaux-Ghee, and F. Sarro, “User-centric deployment of automated program repair at bloomberg,” in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 81–91. [Online]. Available: <https://doi.org/10.1145/3639477.3639756>
- [14] X. Li *et al.*, “Skillsbench: Benchmarking how well agent skills work across diverse tasks,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.12670>
- [15] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “Swe-skills-bench: Do agent skills actually help in real-world software engineering?” 2026. [Online]. Available: <https://arxiv.org/abs/2603.15401>
- [16] Y. Liang *et al.*, “Skillnet: Create, evaluate, and connect ai skills,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.04448>
- [17] Y. Liu, W. Wang, R. Feng, Y. Zhang, G. Xu, G. Deng, Y. Li, and L. Zhang, “Agent skills in the wild: An empirical study of security vulnerabilities at scale,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.10338>
- [18] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, Y. Zhang, and L. Y. Zhang, “Malicious agent skills in the wild: A large-scale security empirical study,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.06547>
- [19] S. Alzubi, N. Provenzano, J. Bingham, W. Chen, and T. Vu, “Evoskill: Automated skill discovery for multi-agent systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.02766>
- [20] J. Gong, R. Gu, Z. Fei, Y. Cao, L. Twist, A. Geiger, S. Han, D. Sobania, F. Sarro, and J. M. Zhang, “Skillmoo: Multi-objective optimization of agent skills for software engineering,” in *41st IEEE/ACM International Conference on Automated Software Engineering - NIER Track*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.09297>